

TRANSLATED BLOG / 译文

从零构建一个最小 AI 智能体

用于软件工程、终端操作及更多任务：从 50 行原型到 mini-swe-agent

作者：Kilian Lieret、Carlos Jimenez、John Yang、Ofir Press

发布日期：网页持续更新

原文：<https://minimal-agent.com>

中文翻译 · DARK READING EDITION

<https://minimal-agent.com>

你想从零开始构建自己的 AI 智能体吗？好消息是：这件事非常简单，尤其是在使用较新的语言模型时。除了调用语言模型所需的包，本教程不会使用其他外部包；最初的最小智能体只有大约 60 行代码。

如果你担心这过于简化、无法用于实际工作：mini-swe-agent 的 mini 智能体正是按同样思路构建的，已经被 Princeton、Stanford、NVIDIA、Anyscale、essentials.ai 等机构用于研究。这个简单方案在 SWE-bench Verified 上最高可达到约 74%，只比高度优化的智能体低几个百分点。

第一个 50 行原型

从最高层看，AI 智能体就是一个大循环：给模型一个提示；智能体提出动作；程序执行动作；把执行结果告诉语言模型；然后重复。为了记录已经发生的事情，我们不断把内容追加到 `messages` 列表。



```
messages = [{"role": "user", "content": "帮我修复 main.py 中的 ValueError"}]
while True:
    lm_output = query_lm(messages)
    print("模型输出", lm_output)
    messages.append({"role": "assistant", "content": lm_output})
    action = parse_action(lm_output)
    print("动作", action)
    if action == "exit":
        break
    output = execute_action(action)
    print("执行输出", output)
    messages.append({"role": "user", "content": output})
```

role 字段是什么？

`role` 表示一条对话消息是谁发送的。常见角色是：

- `"user"`：用户或人类发送的消息。
- `"assistant"`：AI 模型发送的消息。
- `"system"`：用于设置背景和指令的系统提示。

不同语言模型 API 对消息结构的约定可能略有不同。为了让上面的循环真正运行，我们只需要实现三件事：

- 调用语言模型 API。若只使用一个确定的模型，这很简单；如果要支持许多模型或统计详细费用，就会麻烦一些。

- 解析动作 `parse_action`。如果模型支持工具调用，也可以直接使用工具调用功能，不必自己解析；但它更依赖具体供应商，因此本教程暂不展开，而且不用它通常也不会影响性能。
- 执行动作 `execute_action`。这里很简单：把模型提出的动作当作 `Bash` 命令交给终端执行。

调用语言模型

先完成第一步。下面列出几种供应商的最小调用方式。如果你所用的模型未被单独列出，`LiteLLM` 通常是很好的默认选择，因为它支持大量模型供应商。

OpenAI

安装 `OpenAI` 包：

```
pip install openai

from openai import OpenAI

client = OpenAI(
    api_key="your-api-key-here"
) # 也可以设置 OPENAI_API_KEY 环境变量

def query_lm(messages):
    response = client.responses.create(
        model="gpt-5.1",
        input=messages
    )
    return response.output_text
```

Anthropic

```
pip install anthropic

from anthropic import Anthropic

client = Anthropic(
    api_key="your-api-key-here"
) # 也可以设置 ANTHROPIC_API_KEY 环境变量

def query_lm(messages):
    response = client.messages.create(
        model="claude-sonnet-4.5",
        max_tokens=4096,
        messages=messages
    )
    return response.content[0].text
```

OpenRouter

`OpenRouter` 使用与 `OpenAI` 兼容的 `API`：

```
pip install openai

from openai import OpenAI
```

```

client = OpenAI(
    base_url="https://openrouter.ai/api/v1",
    api_key="your-api-key-here"
) # 也可以设置 OPENROUTER_API_KEY 环境变量

def query_lm(messages):
    response = client.chat.completions.create(
        model="anthropic/claude-3.5-sonnet",
        messages=messages
    )
    return response.choices[0].message.content

```

LiteLLM

LiteLLM 支持 100 多种语言模型供应商：

```

pip install litellm

from litellm import completion

def query_lm(messages: list[dict[str, str]]) -> str:
    response = completion(
        model="openai/gpt-5.1", # 可替换为任意供应商和模型
        messages=messages
    )
    return response.choices[0].message.content

```

GLM / 智谱 AI

```

pip install zhipuai

from zhipuai import ZhipuAI

client = ZhipuAI(
    api_key="your-api-key-here"
) # 也可以设置 ZHIPUAI_API_KEY 环境变量

def query_lm(messages):
    response = client.chat.completions.create(
        model="glm-4-plus",
        messages=messages
    )
    return response.choices[0].message.content

```

怎样设置环境变量

不要把 API Key 硬编码在代码中，而应把它设为环境变量。注意：下面的命令默认只对当前终端会话生效，不会永久保存。

Linux / macOS:

```

export OPENAI_API_KEY="your-api-key-here"
export ANTHROPIC_API_KEY="your-api-key-here"
export GOOGLE_API_KEY="your-api-key-here"

```

如需持久保存，可将命令加入 `~/.bashrc`、`~/.zshrc` 或你的 Shell 配置文件。

Windows CMD:

```
set OPENAI_API_KEY=your-api-key-here
set ANTHROPIC_API_KEY=your-api-key-here
set GOOGLE_API_KEY=your-api-key-here
```

Windows PowerShell:

```
$env:OPENAI_API_KEY="your-api-key-here"
$env:ANTHROPIC_API_KEY="your-api-key-here"
$env:GOOGLE_API_KEY="your-api-key-here"
```

要在 Windows 中永久保存，请使用系统属性里的“环境变量”。也可以通过 `python-dotenv` 使用 `.env` 文件。

Python 类型提示

如果你对 `list[dict[str, str]]` 和 `-> str` 感到疑惑：它们是“类型提示”。类型提示在 Python 中不是必需的，但可以帮助 IDE、静态检查器，乃至你自己理解函数的输入和输出。

快速测试

```
messages = [{"role": "user", "content": "掷一个 d20 骰子"}]
print(query_lm(messages))
```

你应该能看到模型的响应，例如一次骰子结果或相应解释。生产实现可以参考 `mini-swe-agent` 中的 `LiteLLM` 和 `OpenRouter` 模型类。

解析动作

模型可以用两种简单方式“编码”动作。如果你使用模型原生工具调用，就不需要这一步；但为了保持教程简单，这里手动解析。

方式一：三反引号代码块（推荐）

模型先解释它准备执行什么，然后给出动作：

```
```bash-action
cd /path/to/project && ls
```
```

方式二：XML 风格标签

模型先解释它准备执行什么，然后给出动作：

```
<bash_action>cd /path/to/project && ls</bash_action>
```

对大多数模型，两种方式都有效；教程推荐三反引号。有些较小或开源模型的泛化能力稍弱，可以分别尝试。下面用正则表达式提取动作。

```
import re
```

```
def parse_action(lm_output: str) -> str:
    """接收模型输出，返回动作"""
    matches = re.findall(
        r"```bash-action\s*\n(.*)\n```",
        lm_output,
        re.DOTALL
    )
    return matches[0].strip() if matches else ""
```

测试三反引号解析方式

```
test_output = """我会列出当前目录中的文件。

```bash-action
ls -la
```

"""

print(parse_action(test_output))
```

XML 风格解析方式

```
import re

def parse_action(lm_output: str) -> str:
    """接收模型输出，返回动作"""
    matches = re.findall(
        r"<bash_action>(.*?)</bash_action>",
        lm_output,
        re.DOTALL
    )
    return matches[0].strip() if matches else ""

test_output = """我会列出当前目录中的文件。

<bash_action>ls -la</bash_action>

"""

print(parse_action(test_output))
```

理解这段正则表达式

- `r"..."`: 原始字符串。`r` 前缀让反斜杠按字面意义处理；否则要把 `\n` 写成 `\\n`，把 `\s` 写成 `\\s`。
- `(.*?)`: 非贪婪捕获组。括号捕获要提取的内容；`.*?` 匹配任意字符，但 `?` 会让它在遇到第一个结束标记时停止，而不是一直匹配到最后一个。
- `re.DOTALL`: 让点号 `.` 也能匹配换行，因此可以捕获多行命令。
- `findall` 只返回括号内捕获的内容，不包含外部标记。

生产实现可以参考 `mini-swe-agent` 的 `default.py` 中的 `parse_action`。

执行动作

执行动作非常简单：使用 Python 的 `subprocess` 模块即可。也可以用 `os.system`，但通常不推荐。

```
import subprocess
import os

def execute_action(command: str) -> str:
    """执行动作并返回输出"""
    result = subprocess.run(
        command,
        shell=True,
        text=True,
        env=os.environ,
        encoding="utf-8",
        errors="replace",
        stdout=subprocess.PIPE,
        stderr=subprocess.STDOUT,
        timeout=30,
    )
    return result.stdout
```

理解 `subprocess.run` 的参数

- `shell=True`：允许执行字符串形式的任意 Shell 命令，包括 `cd`、`ls`、管道等。不要把不可信输入直接交给它。
- `text=True`：以字符串而不是字节返回输出。
- `env=os.environ`：把当前环境变量传递给子进程。
- `encoding="utf-8"`：指定文本输出使用 UTF-8 编码。
- `errors="replace"`：用替代字符处理无效字符，而不是抛出异常。
- `stdout=subprocess.PIPE`：捕获标准输出。
- `stderr=subprocess.STDOUT`：把标准错误重定向到标准输出，于是两者会被同一条输出流捕获。
- `timeout=30`：执行超过 30 秒就停止。

生产实现可参考 `mini-swe-agent` 的环境类：`LocalEnvironment` 最接近上面的代码；`DockerEnvironment` 则把 `subprocess.run` 换成 `docker exec`。

这种实现的两个限制

- 智能体不能通过一次 `cd` 命令永久切换工作目录，因为每次命令都在新的子 Shell 中运行。
- 智能体不容易让环境变量在多次命令之间持续存在。

实践中，这些限制并没有想象中严重。减少隐藏状态、迫使智能体使用绝对路径，反而可能在很多情况下帮助语言模型。`Claude Code` 也有类似特性：它虽可切换目录，却无法让环境变量跨子 Shell 持久存在。

加入系统提示

我们还需要告诉语言模型应该怎样行动：

```
messages = [{
    "role": "system",
    "content": (
        "你是一个乐于助人的助手。想运行命令时，请把它包装成"
        "`bash-action\n<command>\n`。完成任务时，请执行 exit 命令。"
    )
}]
```

把所有部分组合起来并运行

下面是使用 LiteLLM 和三反引号格式的完整最小实现：

```
import re
import subprocess
import os
from litellm import completion

def query_lm(messages: list[dict[str, str]]) -> str:
    response = completion(
        model="openai/gpt-5.1",
        messages=messages
    )
    return response.choices[0].message.content

def parse_action(lm_output: str) -> str:
    """接收模型输出，返回动作"""
    matches = re.findall(
        r"`bash-action\s*\n(?:.*?)\n`",
        lm_output,
        re.DOTALL
    )
    return matches[0].strip() if matches else ""

def execute_action(command: str) -> str:
    """执行动作并返回输出"""
    result = subprocess.run(
        command,
        shell=True,
        text=True,
        env=os.environ,
        encoding="utf-8",
        errors="replace",
        stdout=subprocess.PIPE,
        stderr=subprocess.STDOUT,
        timeout=30,
    )
    return result.stdout

# 智能体主循环
messages = [{
    "role": "system",
    "content": (
        "你是一个乐于助人的助手。想运行命令时，请把它包装成"
        "`bash-action\n<command>\n`。完成任务时，请执行 exit 命令。"
    )
}, {
```



```

        "role": "user",
        "content": "列出当前目录中的文件"
    }]

    while True:
        lm_output = query_lm(messages)
        print("模型输出", lm_output)
        messages.append({"role": "assistant", "content": lm_output})
        action = parse_action(lm_output)
        print("动作", action)
        if action == "exit":
            break
        output = execute_action(action)
        print("执行输出", output)
        messages.append({"role": "user", "content": output})

```

让它更加健壮

下面是一些提升表现的小调整。没有复杂技巧，只是确保智能体不会卡死，并能应对出错情况。这一部分稍微进阶。教程不再重复完整代码，建议直接阅读 `mini` 智能体的源码：它以很少的额外结构实现了这里提到的所有功能。

在控制流程中处理异常

核心思路是：一旦发生已知异常，例如超时或格式错误，就把异常信息告诉语言模型，让它自己处理。只需稍微修改 `while` 循环：

```

while True:
    try:
        # 前面的主要逻辑
        ...
    except Exception as e:
        messages.append({"role": "user", "content": str(e)})

```

就这么简单。例如，智能体做了不合适的事情（如调用 `vim`）并触发 `TimeoutError`，错误信息会被加入 `messages`；语言模型看到后可以继续处理，并有望意识到自己哪里做错了。

你也可以只把特定的已知问题交还给模型，并添加更具体的信息：

```

class OurTimeoutError(RuntimeError):
    ...

def execute_action(action: str) -> str:
    try:
        # 与前面相同
        ...
    except TimeoutError as e:
        raise OurTimeoutError(
            "上一条命令超时了，你可能需要改用非交互式命令……"
        ) from e

```

还可以更精确地区分：哪些异常应交给语言模型，哪些应直接让程序崩溃。定义一个自定义基类，只在循环里捕获该类即可：

```

class NonterminatingException(RuntimeError):
    ...

class OurTimeoutError(NonterminatingException):
    ...

while True:
    try:
        ...
    except NonterminatingException as e:
        ...

```

mini-swe-agent 还定义了 TerminatingException, 用它代替 if action == "exit", 让循环更优雅地停止:

```

class TerminatingException(RuntimeError):
    ...

class Submitted:
    ... # 智能体希望停止

def execute_action(action: str) -> str:
    if action == "exit":
        raise TerminatingException("语言模型请求退出")
    ...

while True:
    try:
        ...
    except NonterminatingException as e:
        ...
    except TerminatingException as e:
        print("停止原因: ", str(e))
        break

```

处理格式错误的输出

语言模型有时（尤其是能力较弱的模型）不会正确格式化动作。这时应提醒它采用正确格式。既然已经有通用异常处理，实现起来很直接：

```

incorrect_format_message = """你的输出格式不正确。
请严格包含 1 个动作，并按下面的示例格式书写：

```bash-action
ls -R
```
"""

class FormatError(RuntimeError):
    ...

def parse_action(action: str) -> str:
    matches = ...
    if not len(matches) == 1:
        raise FormatError(incorrect_format_message)
    ...

```

环境变量

可以设置一些环境变量来禁用命令行工具的交互式界面，从而避免智能体卡住。mini-swe-agent 的 SWE-bench 配置也采用了类似设置：

```
env_vars = {
    "PAGER": "cat",
    "MANPAGER": "cat",
    "LESS": "-R",
    "PIP_PROGRESS_BAR": "off",
    "TQDM_DISABLE": "1",
}

# ...
def execute_action(command: str) -> str:
    # ...
    result = subprocess.run(
        command,
        # ...
        env=os.environ | env_vars
        # ...
    )
```

mini-swe-agent

mini-swe-agent 完全按照本教程的蓝图构建，源码应该很容易读懂。唯一需要注意的是，它采用了更模块化的结构，方便替换各个组件。

Agent 类的 run 函数中包含那个核心大循环：

```
class Agent:
    def __init__(self, model, environment):
        self.model = model
        self.environment = environment
        ...

    def run(self, task: str):
        while True:
            ...
```

Model 类负责不同语言模型：

```
class Model:
    def query(messages: list[dict[str, str]]):
        ...
```

Environment 类负责执行动作：

```
class Environment:
    def execute(command: str):
        ...
```

mini-swe-agent 提供多种环境类，例如让动作在 Docker 容器而不是本地环境执行。听起来更复杂，但实质上只是把 subprocess.run 换成对 docker exec 的调用。

为本指南贡献内容

项目欢迎通过 [GitHub](#) 改进本指南。错误修复和拼写修复会立即合并；新增尚未涵盖的热门模型支持也很受欢迎，通常会较快合并，但请确保测试实现。新增章节或大幅扩写则应先通过 [GitHub Issue](#) 讨论。改动越大，审查所需时间通常越长；如果没有预先讨论，被接受的可能性也会降低。

- Fork 仓库。
- 完成修改。
- 提交 Pull Request。

源码可在 [GitHub](#) 上找到。如有问题或意见，可以在页面下方留言；Bug 报告和后续开发讨论仍优先使用 [GitHub Issues](#)。

原文链接: <https://minimal-agent.com>

说明：本 PDF 为中文翻译与重新排版版本。为保证示例可运行，Python 标识符、命令、API 字段与模型名称保留原文；解释、标题、注释和可读提示语已译为中文。